# Lab: Exploring a Flask Server and REST API with Hangman

CSE 4095 Agentic Programming
Alexander Grejdus (ajg22021) · 2026-10-22

## Part 1: Explore the Hangman Web Application (15 pts)

### Q1 (5 pts)

*Identify three components of the Hangman application: the client, the server, and the game engine. Explain the responsibility of each component.*

**Client:**

The Client part of this hangman application is the guessing aspect, and the visual components the user can see. The client is sent information about current game state, letters guessed and missed, leaderboard etc. The client can also send a letter guess to the server.

**Server:**

The server is the main part bridging the game engine to the client. When the client sends a letter guess, the server receives these HTTP requests at the API endpoints (like POST /api/games/<id>/guess). The server then find the right game by its ID, hands the guess to the engine, and sends the result back as a JSON file.

**Game engine:**

The game engine here controls the logic of the game. What word is generated, what the difficulties mean when the client clicks them. The game engine also implements the core rules such as number of lives, amount of guesses allowed, and decides if the game is won or lost.

### Q2 (5 pts)

*Why should the hidden word be maintained by the server instead of being sent to the browser when the game starts?*

The hidden word should be maintained by the server, because if the word is sent to the browser, then the client can access the word before making any guesses. Anyone can read the word by inspecting element to see developer tools information.

### Q3 (5 pts)

*How can the server distinguish between two games being played by different users?*

The server distinguishes these two different users by communicating via the game ID. Each new game has a unique game id specific to that game. Game ID looks something like this: 42cdd4659f37

## Part 2: Inspect HTTP Requests (20 pts)

### Q4 (5 pts)

*Record the request used to create the game.*

**Request URL:** `https://hangman-ex89.onrender.com/api/games`

**HTTP method:** `POST`

**Request body (if present):**

```
{"difficulty": "easy"}
```

**HTTP response status code:** `201 Created`

### Q5 (5 pts)

*Examine the response body. Identify the game identifier and at least three other fields returned by the server. Explain what each field represents.*

**Response body (paste JSON):**

```
{"game_id": "f8573706113d", "pattern": "_ _ _ _ _", "remaining_guesses": 6, "status": "playing"}
```

**Field explanations:**

guessed letters is the letters the client has guessed so far.
incorrect_letters are the incorrect letters the client has guessed, so correct guesses do not appear here.
pattern is the current pattern the client sees
remaining guesses is the amount of guesses the client has remaining before losing
status is either playing, won, or lost.
game identifier is the game_id, which is in /api/games/game_id endpoint

### Q6 (5 pts)

*Record the endpoint, HTTP method, and JSON request body used to submit the guess.*

**Endpoint:** `/api/games/f8573706113d/guess`

**HTTP method:** `POST`

**JSON request body:**

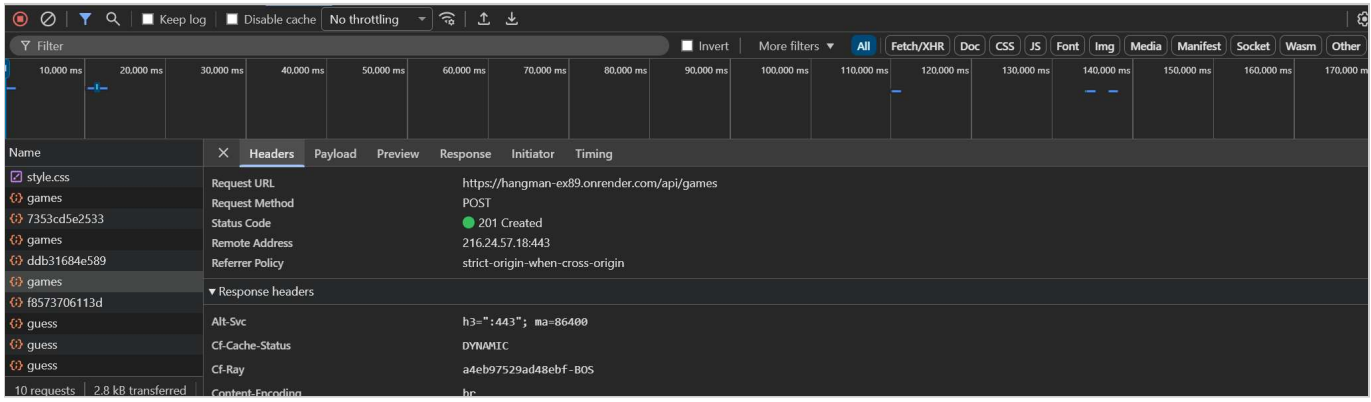
```
{"letter": "e"}
```

### Q7 (5 pts)

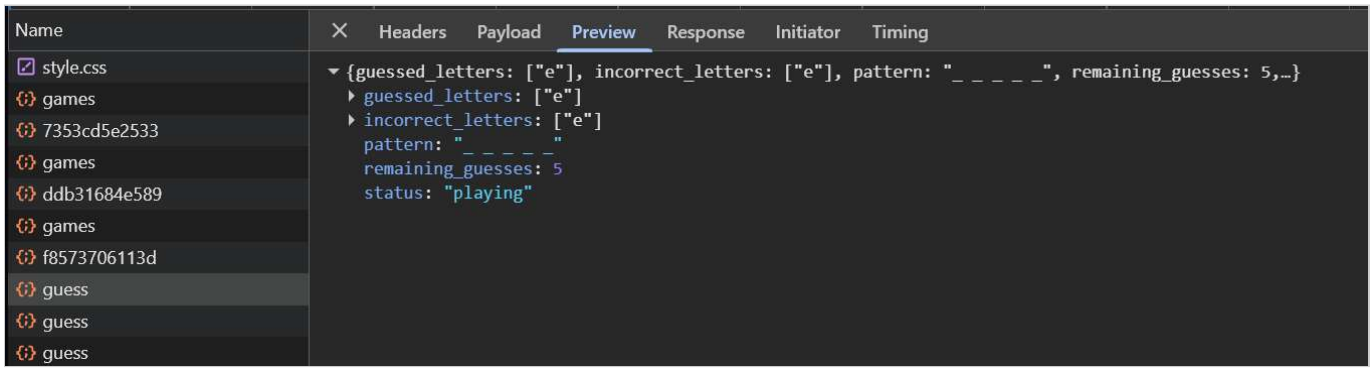
*Compare the server responses before and after the guess. Identify at least two fields that changed, or explain why a field remained unchanged.*

When the request is made to the server, and the server sends back the information, two fields that change are "guesses letters", "incorrect letters", and "remaining guesses. Guessed letters and incorrect letters both append the guess the client made, and remaining guesses decrements by 1.

### Evidence




Creating a game (create_game.png)




Submitting a guess (guess_letter.png)

# Part 3: Interact with the REST API Using curl (20 pts)

## Evidence

### Task 3.1: Create a game:

```
{
    "game_id": "9fde283b77e7",
    "pattern": "_ _ _ _ _",
    "remaining_guesses": 6,
    "status": "playing"
}
```

### Task 3.2: Submit a guess:

```
{
    "guessed_letters": [
        "e"
    ],
    "incorrect_letters": [],
    "pattern": "_ _ _ e",
    "remaining_guesses": 6,
    "status": "playing"
}
```

### Task 3.3: Retrieve the game state:

```
{
    "guessed_letters": [
        "e"
    ],
    "incorrect_letters": [],
    "pattern": "_ _ _ e",
    "remaining_guesses": 6,
    "status": "playing"
}
```

### Q8 (5 pts)

*Explain the difference between the GET and POST requests used in this exercise.*

In this exercise, the POST method is the client sending information to the server. In this case it's sending the information that a new request to make a game was created, or that a new request to make a guess was made.

The GET method, is the client asking to read the game state, which the server ends up sending back information to the client. So when we make the request to retrieve the game state, this is a GET method.

### Q9 (5 pts)

*What is the purpose of the Content-Type: application/json header?*

The purpose of this Content-Type header is to let the server know that the request body is in the proper JSON format. Without the header, curl would send the data as url encoded form data, and the program might not be able to properly parse it.

### Q10 (5 pts)

*What happens when you request a game identifier that does not exist? Record the HTTP status code and explain its meaning.*

**Command and output:**

```
{
    "error": "unknown game"
}
```

**Status code:** `404 Not Found`

**Meaning:**

404 means the url was a valid url, but the server had no game stored under that ID at that address.

### Q11 (5 pts)

*Explain why a terminal application can interact with the Hangman server without using the web interface.*

A terminal can interact with the Hangman server, because the server exposes a REST API over HTTP. The web interface uses HTML to create the buttons, and javascript sends these requests when

the button is clicked, abstracted as a simple "New Game" button, instead of say, a curl command. Anything that can send HTTP methods and JSONs can interact with the hangman game through the REST API.

## Part 4: Develop a Simple Python REST API Client (30 pts)

### Q12 (10 pts)

*Submit your working hangman_client.py program.*

Submitted as a separate file: `hangman_lab/hangman_client.py`.

### Q13 (10 pts)

*Demonstrate one complete game played through your Python client.*

```
ajgre@WIN-MGHE7JJ91UU:~/CSE4095_2026/in_class_assignments/class14-hangmanlab/hangman_lab$ python3 hangman_client.py
Connecting to the Hangman server (this can take up to a minute if it is asleep)...
{'game_id': '63414d2a825e', 'pattern': '_ _ _ _', 'remaining_guesses': 6, 'status': 'playing'}

New game created! Game ID: 63414d2a825e

----------------------------------------
word:               _ _ _ _
remaining guesses:  6
guessed letters:    none
incorrect letters:  none
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: a

----------------------------------------
word:               _ _ _ _
remaining guesses:  5
guessed letters:    a
incorrect letters:  a
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: z

----------------------------------------
word:               _ _ _ _
remaining guesses:  4
guessed letters:    a, z
incorrect letters:  a, z
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: x

----------------------------------------
word:               _ _ _ _
remaining guesses:  3
guessed letters:    a, z, x
incorrect letters:  a, z, x
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: c

----------------------------------------
word:               _ _ _ _
remaining guesses:  2
guessed letters:    a, z, x, c
incorrect letters:  a, z, x, c
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: v

----------------------------------------
word:               _ _ _ _
remaining guesses:  1
guessed letters:    a, z, x, c, v
incorrect letters:  a, z, x, c, v
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: q

----------------------------------------
word:               _ _ _ _
remaining guesses:  0
guessed letters:    a, z, x, c, v, q
incorrect letters:  a, z, x, c, v, q
status:             lost
----------------------------------------
```




Python client screenshot (python_client.png)

## Terminal transcript (optional):

```
ajgre@WIN-MGHE7JJ91UU:~/CSE4095_2026/in_class_assignments/class14-hangmanlab/hangman_lab$ python3
hangman_client.py
Connecting to the Hangman server (this can take up to a minute if it is asleep)...
{'game_id': '63414d2a825e', 'pattern': '_ _ _ _', 'remaining_guesses': 6, 'status': 'playing'}

New game created! Game ID: 63414d2a825e

----------------------------------------
word:               _ _ _ _
remaining guesses:  6
guessed letters:    none
incorrect letters:  none
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: a

----------------------------------------
word:               _ _ _ _
remaining guesses:  5
guessed letters:    a
incorrect letters:  a
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: z

----------------------------------------
word:               _ _ _ _
remaining guesses:  4
guessed letters:    a, z
incorrect letters:  a, z
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: x

----------------------------------------
word:               _ _ _ _
remaining guesses:  3
guessed letters:    a, z, x
incorrect letters:  a, z, x
status:             playing
----------------------------------------
_ _ _ _
Enter a letter to guess: c

----------------------------------------
word:               _ _ _ _
remaining guesses:  2
guessed letters:    a, z, x, c
```

```
incorrect letters:  a, z, x, c
status:             playing
----------------------------------------

_ _ _ _
Enter a letter to guess: v

----------------------------------------
word:               _ _ _ _
remaining guesses:  1
guessed letters:    a, z, x, c, v
incorrect letters:  a, z, x, c, v
status:             playing
----------------------------------------

_ _ _ _
Enter a letter to guess: q

----------------------------------------
word:               _ _ _ _
remaining guesses:  0
guessed letters:    a, z, x, c, v, q
incorrect letters:  a, z, x, c, v, q
status:             lost
----------------------------------------

 You Lost! The word was :  mind
```

**Q14 (10 pts)**

*Explain how your Python client interacts with the Flask server. Specifically, describe the role of HTTP requests, JSON, and the game identifier.*

**HTTP requests:**

The only HTTP REquests made are through these functions:

```
@handle_errors
def create_game(difficulty):
    payload = {"difficulty":difficulty}
    r = requests.post(POST_URL, json = payload)

    return ast.literal_eval(r.text)



@handle_errors
def get_game(game_id):
    r = requests.get(f"{GET_URL}{game_id}");

    return ast.literal_eval(r.text)

@handle_errors
def make_guess(game_id, letter):
```

```
    payload = {"letter":letter}
    r = requests.post(f"{GET_URL}{game_id}/guess", json = payload);

    return ast.literal_eval(r.text)
```

These functions create the proper payload the request is expecting, then I either GET or POST to the proper link.

**JSON:**
The JSON for this game keeps track of all the data received from the server, and the data sent from the client. When create a gaming, I send a json with the difficulty, and the server sends back a json of the game_id, and the rest of the game states.

**Game identifier:**
The identifier of the game is the game_id. I get this from the response from the POST request made to create the game. The client puts the game_id in the URL of every later request, such as sending a letter or getting game state.

## Part 5: Reflection: Agentic Programming (15 pts)

**Q15 (5 pts)**
*What tasks did you use an AI coding assistant for? Give at least two specific examples of prompts you used.*

**Tasks:**
I used AI to help explain why the syntax of my POST requests was not working. I also used AI to implements part 9 which is the error and exception handling, as I do not know it very well in python. Other than that, everything was programmed by hand. Both of the responses I got was just syntax that I was missing, and proper error handling methods in python which I do not know.
I also used AI to review responses to the questions, just to make sure my ideas were right. I slightly tweaked answers based on that.

**Prompt example 1:**

```
4370af9dddc7
Traceback (most recent call last):
  File "/home/ajgre/CSE4095_2026/in_class_assignments/class14-
hangmanlab/hangman_lab/hangman_client.py", line 36, in <module>
    game = get_game(game_id)
           ^^^^^^^^^^^^^^^^^
  File "/home/ajgre/CSE4095_2026/in_class_assignments/class14-
hangmanlab/hangman_lab/hangman_client.py", line 16, in get_game
    r = requests.get(GET_URL.join(game_id));
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/usr/lib/python3/dist-packages/requests/api.py", line 73, in get
    return request("get", url, params=params, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/requests/api.py", line 59, in request
    return session.request(method=method, url=url, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/requests/sessions.py", line 589, in request
    resp = self.send(prep, **send_kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/requests/sessions.py", line 697, in send
    adapter = self.get_adapter(url=request.url)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/requests/sessions.py", line 794, in get_adapter
    raise InvalidSchema(f"No connection adapters were found for {url!r}")
requests.exceptions.InvalidSchema: No connection adapters were found for '4https://hangman-
ex89.onrender.com/api/games/3https://hangman-
ex89.onrender.com/api/games/7https://hangman-
ex89.onrender.com/api/games/0https://hangman-
ex89.onrender.com/api/games/ahttps://hangman-
ex89.onrender.com/api/games/fhttps://hangman-
ex89.onrender.com/api/games/9https://hangman-
ex89.onrender.com/api/games/dhttps://hangman-
ex89.onrender.com/api/games/dhttps://hangman-
ex89.onrender.com/api/games/dhttps://hangman-
ex89.onrender.com/api/games/chttps://hangman-ex89.onrender.com/api/games/7'
```

**Prompt example 2:**

[pasted code]
Please only ADD lines to this code, add the necessary error handling and printing game state

### Q16 (5 pts)

*Describe one problem you encountered while developing or testing your Python client. How did you diagnose and resolve it?*

**Problem:**

One problem I encountered when setting up the client, was that my get methods and post methods were not working.

**Diagnosis:**

Searching google for requests library syntax, and asking Opus the problems with the code I had, for the errors I was getting.

**Resolution:**

I fixed the syntax to follow this:
r = requests.post(POST_URL, json = payload)
before, I had it as params = payload, instead of json, which was giving me an error when I tried to post.

### Q17 (5 pts)

*Suppose you want to replace the human player with an automated Hangman bot. What changes would you make to your Python client? Would the Flask server need to change? Explain.*

**Client changes:**

The only changes I would make are for the guesses to be automated via an algorithm. Making a bot, a simple bot would be to check a dictionary for the most common letters in english, and guess those letters one at a time. IT would do the same requests and checking game state, it just wouldn't need to constantly visually see the guesses made and game state.

**Server changes?:**

The flask server would not need to change, because it's basically the same as the client typing in letters, but an automated check. The bot sends the same HTTP requests and JSON as a human does, as the API doesn't care what type of client is calling it.